Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- Dersin tarihi 18 Kasım 2020 Çarşamba (7. hafta). Ders Zoom üzerinden yapıldı.
  - Asıl anlatım yaklaşık 00:19:40'ta başlar. İlk 18 dakika vize, ödev ve sınav kuralları hakkındaki duyurulardır.
  - 00:56:00 ile 01:07:40 arası ara.
  - Geçen haftaki ders yarım kaldığı için bu haftanın uygulama saati de konu anlatımına ayrıldı (ders 14).
- Vize duyurusu
- Hashing tekrarı
- Çarpma yöntemi (multiplication method)
- Doğum günü paradoksu (birthday paradox) ve çakışma olasılığı
- Anahtar türleri: tam sayı, kesirli sayı, string, bileşik anahtar (compound key)
  - String'den anahtar üretme ve Horner yöntemi
- Çakışma çözümleri
  - Ayrık zincirleme (separate chaining, open hashing)
  - Açık adresleme (open addressing, closed hashing)
- Açık adresleme yöntemleri
  - Doğrusal sınama (linear probing)
  - İkinci dereceden sınama (quadratic probing)
  - Çift hashing (double hashing)
- Arama döngüsünün sonlanması, tablonun dolması, yeniden hashleme (rehashing), silme işareti

Hocanın Özellikle Vurguladığı Kısımlar

- Vize yüz yüze vizelerle aynı zorlukta olacak.
  - Test, boşluk doldurma, algoritma tasarımı ve analiz soruları olabilir.
  - Daha önce görmediğiniz bir soruyla karşılaşabilirsiniz.
- Dönem içi ödevleri mutlaka yapın.
  - Geçen dönem 70 kişiden 20'si hiç ödev yapmamıştı.
- Hash fonksiyonu kolay hesaplanabilir olmalıdır.
  - Çarpma yöntemi bölme yönteminden daha çok işlem yapar. Bu yüzden pratikte daha az kullanılır.
- String anahtarlarda karakter kodlarını basitçe toplamak kötü bir yöntemdir.
  - Aynı harflerden oluşan sözcükler (harflerin yeri değişmiş sözcükler) aynı toplamı verir. Derste verilen örnek sözcükler altyazıda bozuk geçiyor.
- Doğrusal sınamada arama döngüsüne mutlaka "$i < m$" koşulu eklenmelidir. Yoksa tablo doluyken arama sonsuz döngüye girer.
- Açık adreslemede tablo yüzde 80-90 dolmaya yaklaşınca yeniden hashleme (rehashing) yapılır.
  - Daha büyük bir tablo açılır ve elemanlar yeni hash fonksiyonuyla yeni tabloya yerleştirilir. Elemanlar aynı konuma düşmeyebilir.
- Doğrusal sınamanın sorunu öbeklenmedir (clustering).

Kısa Tekrar Notları

- Yük faktörü (load factor): $\alpha = n/m$. n eleman sayısı, m tablo boyutu.
- Çarpma yöntemi: $h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$, $0 < A < 1$.
  - En çok kullanılan değer $A = \frac{\sqrt{5}-1}{2} \approx 0.6180339887$.
  - Örnek: $m = 10000$, $k = 123456 \Rightarrow kA \approx 76300.0041151$, kesir kısmı 0.0041151, $h(k) = 41$.
- k anahtarın hiç çakışmama olasılığı: $\frac{m-1}{m} \cdot \frac{m-2}{m} \dots \frac{m-(k-1)}{m} \approx e^{-k(k-1)/(2m)}$.
- Çakışma olasılığı: $p \approx 1 - e^{-k(k-1)/(2m)}$.
  - $m = 2^{32}$ için yaklaşık 77.163 eklemeden sonra $p \approx 0.5$.
  - Yaklaşık 250.000 eklemeden sonra $p \approx 0.999$.
- String anahtar (yöntem A): $h = 0$; her karakter için $h = h \cdot C + \text{ord}(c_i)$. C, karakter kodlarından büyük bir sabit.
- Horner yöntemi: $h = h \cdot R + (\text{str}[i] - \text{'a'} + 1)$, genelde $R = 31$ (asal).
- Bileşik anahtar örneği (31.08.1997): $31 \cdot R + 8 \cdot R^2 + 1997 \cdot R^3$.
- Doğrusal sınama: $h(k, i) = (h'(k) + i) \bmod m$.
- İkinci dereceden sınama: $h(k, i) = (h'(k) + i^2) \bmod m$ ya da $(h'(k) + c_1 i + c_2 i^2) \bmod m$.
- Çift hashing: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$, $h_1(k) = k \bmod m$, $h_2(k) = 1 + (k \bmod m')$, $m' < m$.
- Silinen elemanın yerine bir silme işareti (örneğin -1) konur. Silme çok artarsa tablo yeniden düzenlenir.

Detaylı Açıklamalar

Vize duyurusu

- 8. hafta vize haftasıdır. Bu dersin vizesi, dersin olduğu gün saat 15:00'te olacak (25 Kasım 2020 Çarşamba).

Çarpma yöntemi

$$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$$

$kA \bmod 1$, kA'nın kesir kısmıdır: $kA - \lfloor kA \rfloor$. Bu değer 0 ile 1 arasındadır. Tablo boyutu m ile çarpılıp tam kısmı alınır.

Örnek: $m = 10000$, $k = 123456$, $A = 0.6180339887$.

- $kA = 76300.0041151\dots$
- Kesir kısmı: 0.0041151
- $m \cdot 0.0041151 = 41.151 \Rightarrow h(123456) = 41$

Derste bir öğrenci "anahtar büyükse adres tablo boyutunu geçebilir mi?" diye sordu. Geçemez. Kesir kısmı 1'den küçük olduğu için sonuç her zaman m'den küçüktür.

Bölme yöntemiyle karşılaştırma: Çarpma yönteminde daha çok işlem vardır. Bu yüzden "kolay hesaplanabilirlik" açısından bölme yöntemi öndedir. Çarpma yöntemi biraz daha düzgün dağılım verebilir. Karşılaştırma yaparken iki yöntemde de m aynı seviyede seçilmelidir.

Doğum günü paradoksu ve çakışma olasılığı

Soru: Bir ortamdaki k kişinin doğum günlerinin hepsinin farklı olma olasılığı nedir?

- 2 kişi: $\frac{364}{365}$
- 3 kişi: $\frac{364}{365} \cdot \frac{363}{365}$
- k kişi: $\frac{364}{365} \cdot \frac{363}{365} \dots \frac{365-(k-1)}{365}$

Hash tablosuna uyarlanınca 365 yerine tablo boyutu m gelir. k anahtarın hiç çakışmama olasılığı:

$$\prod_{i=1}^{k-1} \frac{m-i}{m} \approx e^{-k(k-1)/(2m)}$$

Çakışma olasılığı bunun 1'den çıkarılmasıdır. $m = 2^{32}$ (32 bitlik hash değerleri) için:

- $k = 77163 \Rightarrow p \approx 0.5$
- $k = 250000 \Rightarrow p \approx 0.999$ (pratikte %100)

Anahtar türleri

1. Tam sayı: doğrudan kullanılır.
2. Kesirli sayı: önce tam sayıya çevrilmelidir. Bu dönüşüm bizim sorumluluğumuzdadır.
3. String: Karakterlerin ASCII kodları toplanırsa aynı harflerden oluşan sözcükler aynı sayıyı verir. Bunun yerine konuma bağlı bir yöntem kullanılır.
   - Yöntem A: $h = 0$ ile başlanır. Her karakterde $h = h \cdot C + \text{ord}(c_i)$. C, string'deki bütün karakter kodlarından büyük bir sabittir.
   - Horner yöntemi: Her karakterde $h = h \cdot R + (\text{str}[i] - \text{'a'} + 1)$. Alfabenin ilk harfi 1 değerini alır. R genelde asal seçilir, sık kullanılan değer 31'dir.
   - Elde edilen sayı sonra bölme ya da çarpma yöntemine verilir.
4. Bileşik anahtar: Doğum günü (31.08.1997) ya da yaş-kilo-boy gibi birden fazla değerden tek anahtar üretilir: $31 \cdot R + 8 \cdot R^2 + 1997 \cdot R^3$.

Çakışma çözümleri

- **Ayrık zincirleme (separate chaining, open hashing):** Aynı adrese düşen elemanlar o gözden başlayan bir bağlı listede tutulur. Ders 14'te işlendi.
- **Açık adresleme (open addressing, closed hashing):** Hesaplanan göz doluysa tabloda bir sonraki uygun göze bakılır.

"Open hashing" ile "open addressing" adları karışabilir. Hoca bu yüzden "separate chaining" ve "open addressing" adlarının kullanılmasını önerdi.

Doğrusal sınama (linear probing)

$$h(k, i) = (h'(k) + i) \bmod m, \quad i = 0, 1, 2, ...$$

Örnek ($m = 7$):

Anahtar	k mod 7	Yerleştiği göz
9	2	2
16	2 (dolu)	3
11	4	4
12	5	5
13	6	6
23	2 (dolu)	3, 4, 5, 6 dolu, mod ile başa döner: 0
8	1	1

Bu noktada tablo doludur. Yeni bir anahtar (örneğin 28 ya da 30) yerleştirilemez.

Arama: $i = 0$'dan başlanır. Göz doluysa ve aranan anahtar değilse i artırılır. Anahtar bulunursa arama başarılıdır. Boş göz (NULL) görülürse anahtar tabloda yoktur.

Tablo tamamen doluysa ve anahtar yoksa döngü hiç bitmez: ne anahtar ne boş göz bulunur. Bu yüzden döngüye $i < m$ koşulu eklenir. Ekleme işleminde de $i = m$ olursa "tablo dolu" hatası verilir.

Tablonun dolması: Asıl çözüm tablonun dolmasını beklememektir. Tablo yaklaşık %80-90 dolunca yeniden hashleme yapılır: daha büyük bir tablo açılır, elemanlar yeni hash fonksiyonuyla yeniden yerleştirilir.

Silme: Silinen gözü boş bırakmak aramayı bozar, çünkü arama boş gözde durur. Bu yüzden göze bir silme işareti (örneğin -1) konur. Silmeler çok artarsa tablo yeniden düzenlenir; gerekirse yeni bir tablo açılır.

Öbeklenme (clustering): $+1$ adımlarla ilerlendiği için yakın adreslere düşen anahtarlar arka arkaya birikir. Tablo düzgün dolmaz.

İkinci dereceden sınama (quadratic probing)

$$h(k,i) = (h'(k) + i^2) \bmod m$$

Adımlar $+1, +4, +9, \ldots$ olur. Böylece elemanlar tabloya daha geniş yayılır.

Örnek ($m = 7$, 2 ve 3 numaralı gözler dolu), $k = 9$:

- $i = 0$: $9 \bmod 7 = 2$ (dolu)
- $i = 1$: $(2 + 1) \bmod 7 = 3$ (dolu)
- $i = 2$: $(2 + 4) \bmod 7 = 6 \Rightarrow$ 9, 6 numaralı göze yerleşir.

Genel biçim: $h(k,i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$. Örnek: $c_1 = 2$, $c_2 = 1$, $i = 1$ için $(2 + 2 + 1) \bmod 7 = 5$.

Aynı adrese düşen anahtarlar yine aynı sırayı ($+1, +4, +9$) izler. Ama farklı adreslere düşen anahtarların dizileri birbirinden uzaklaşır. Toplamda tablo daha dengeli dolar.

Çift hashing (double hashing)

$$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

- $h_1(k) = k \bmod m$
- $h_2(k) = 1 + (k \bmod m')$, $m' < m$

Örnek: $m = 13$, $h_1(k) = k \bmod 13$, $h_2(k) = 1 + (k \bmod 11)$. Tabloda büyük olasılıkla (slayttan) 1. gözde 79, 4. gözde 69, 5. gözde 58, 7. gözde 72 var. $k = 14$ eklenecek:

- $h_1(14) = 1$, $h_2(14) = 1 + 3 = 4$
- $i = 0$: göz 1 (dolu)
- $i = 1$: $(1 + 4) \bmod 13 = 5$ (dolu)
- $i = 2$: $(1 + 8) \bmod 13 = 9 \Rightarrow$ 14, 9 numaralı göze yerleşir.

Adım büyüklüğü anahtara bağlı olduğu için aynı adrese düşen anahtarlar bile farklı dizileri izler. Daha maliyetlidir ama örüntüyü (pattern) bozmakta daha etkilidir.

Ders sonrası notlar

- A **değeri:** Hoca A'yı "0.618033" olarak söyledi. Verilen 76300.0041151 sonucu A'nın tam değeriyle ($0.6180339887\ldots$) elde edilir. Altı basamaklı değerle çarpım 76299.882 olur ve adres değişir (Python ile kontrol edildi).
- **Çakışma formülündeki 2:** Tahtaya $1 - e^{-k(k-1)/m}$ yazıldı. Doğrusu $1 - e^{-k(k-1)/(2m)}$'dir. Derste verilen "77.163 eklemede %50" sonucu 2'li formülden gelir. 2'siz formül 77.163 için yaklaşık 0,75 verir.
- **250.000 eklemede %100 neden?** Derste bir öğrenci "tablo doldu" diye açıkladı, hoca da kabul etti. Bu yanlış. $m = 2^{32} \approx 4{,}3$ milyar gözdür. 250.000 anahtar tablonun yalnız yaklaşık %0,006'sını doldurur. Olasılığın 1'e yaklaşmasının nedeni doğum günü paradoksudur: çakışma olasılığı k ile değil k^2 ile büyür. Kabaca $k \approx \sqrt{m}$ civarında çakışma beklenir hâle gelir ($\sqrt{2^{32}} = 65.536$).
- **Doğum günü sorusu:** Bir öğrencinin "iki kişinin doğum günü aynı olma olasılığı $1/365$" cevabı doğrudur. Derste bu cevap revize edilmek istendi; ama soru tam bu iki kişi içindir.
- **Örnekteki** m**:** Doğrusal sınama örneğinde hoca önce "$m = 6$" dedi, sonra $m = 7$ ile hesapladı. Doğrusu $m = 7$.
- **Tahtadaki etiket:** İkinci dereceden sınama örneğinde üçüncü satır "hash(9,1)" yazılmış. Doğrusu "hash(9,2)"; derste bir öğrenci de düzeltti.

- **Çift hashingde h_2 neden m'den küçük?** Hoca "k m'den büyük olursa tablonun dışında değer üretir" dedi. Bu tam doğru değil, çünkü sonuçta mod$m$ alındığı için adres hep tablonun içindedir. Asıl nedenler şunlardır:
  - $h_2(k)$ hiçbir zaman 0 (ya da m'nin katı) olmamalıdır. Yoksa adım büyüklüğü sıfır olur ve hep aynı göze bakılır. $h_2(k) = 1 + (k \bmod m')$ ve $m' < m$ seçilince $1 \leq h_2(k) \leq m - 1$ olur.
  - $h_2(k)$ ile m aralarında asal olmalıdır. Böylece sınama dizisi bütün gözleri gezer. m asal seçilirse bu koşul kendiliğinden sağlanır.
- **"Separate chaining tablo dolunca kullanılır" cevabı:** Hoca bunu kabul edilebilir saydı. Ayrık zincirleme bir alternatif yöntemdir. Açık adreslemeyle devam ediliyorsa doğru çözüm yeniden hashlemedir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.